MAINCRAFTS TECHNOLOGY

www.maincrafts.com | hr@maincrafts.com

Introduction to Embedded Systems & IoT Device Design

EMBEDDED SYSTEMS TASK – 1

Mini Project

Smart Temperature Monitor

Submitted By	Matru Prasad Behera
Registration Number	24E102C21
Branch	Electrical and Electronics Engineering (EEE)
Semester	4th Semester
Subject	Introduction to Embedded Systems & IoT Device Design
College / University	Institute of Technical Education and Research (ITER), Siksha 'O' Anusandhan
Organisation	Maincrafts Technology

Table of Contents

1. Introduction to Embedded Systems

1.1 What is an Embedded System?

An embedded system is a combination of computer hardware and software — and often additional mechanical parts — designed to perform one dedicated function within a larger device, in contrast to a general-purpose computer that is built to run many different applications. It typically consists of a microcontroller or microprocessor, memory, input devices such as sensors or switches, output devices such as displays, motors, or LEDs, and firmware that continuously reads inputs and drives outputs to carry out its task.

Most embedded systems share a common set of characteristics:

- **Dedicated Functionality:** built to do one job, and to do it reliably, rather than run arbitrary general-purpose software.
- **Real-Time Response:** many embedded systems must react to inputs within a strict, predictable time limit.
- **Resource-Constrained:** limited processing power, memory, and storage compared with a desktop computer.
- **Low Power Consumption:** especially important for battery-powered or always-on devices.
- **Tight Hardware–Software Integration:** the firmware is written specifically for the exact hardware it controls.

1.2 Real-World Applications

Embedded systems are present in almost every modern device. Some of the most common application areas include:

- **Smart Home:** smart thermostats, smart plugs and bulbs, video doorbells, and voice-controlled assistants.
- **Automotive:** engine control units (ECU), anti-lock braking systems (ABS), airbag controllers, and infotainment systems.
- **Healthcare:** patient monitors, pacemakers, infusion pumps, glucose monitors, and wearable fitness trackers.
- **Robotics:** industrial robotic arms, autonomous mobile robots, and drones.
- **Consumer Electronics:** washing machines, microwave ovens, digital cameras, and smart TVs.
- **Industrial Automation:** programmable logic controllers (PLCs), SCADA sensor nodes, and factory-floor monitoring systems.

1.3 Microcontroller vs Microprocessor

The terms microcontroller and microprocessor are often used interchangeably, but they refer to two different kinds of chips:

Parameter	Microcontroller	Microprocessor
Definition	A complete computing system (CPU,	Only a central processing unit on a

Parameter	Microcontroller	Microprocessor
	RAM, ROM, I/O) integrated on a single chip	chip; needs external RAM, ROM, and I/O chips
Core Components	CPU + Memory + I/O ports, all built-in	CPU only
Cost	Lower, for a complete embedded solution	Higher once supporting chips are added
Power Consumption	Low	Comparatively higher
Processing Power	Moderate — sufficient for dedicated tasks	High — suited to running full operating systems
Typical Use	Embedded, single-purpose devices	General-purpose computing (PCs, laptops, servers)
Examples	Arduino (ATmega328P), ESP32, PIC	Intel Core i5/i7, AMD Ryzen

In short, a microcontroller suits dedicated, single-purpose designs such as the temperature monitor built in this task, while a microprocessor is chosen when a system must run a full operating system alongside multiple concurrent applications.

2. Component Study

2.1 Role of Microcontrollers

Arduino (Uno)

The Arduino Uno is an open-source development board built around the 8-bit Microchip ATmega328P microcontroller. It provides 14 digital I/O pins (6 with PWM output), 6 analog input pins, a USB interface for programming and serial communication, and runs at 16MHz. Its simple C/C++-based Arduino IDE, large library ecosystem, and low cost make it the most common starting point for embedded and IoT prototyping — and it is the board used to build the mini project in this report.

ESP32

The ESP32 is a more powerful system-on-chip built around a dual-core Xtensa LX6 processor, with built-in Wi-Fi and Bluetooth radios. This on-board wireless connectivity makes it especially popular for IoT projects that need to send sensor data to the cloud, a mobile app, or a local network without any additional networking hardware. It also offers more GPIO pins, multiple ADC channels, and lower-power sleep modes than the Uno.

Raspberry Pi Pico

The Raspberry Pi Pico is built around the RP2040 chip, which houses a dual-core ARM Cortex-M0+ processor. It can be programmed in either MicroPython or C/C++, and its Programmable I/O (PIO) block allows it to implement custom digital communication protocols directly in hardware. It sits between the Uno and the ESP32 in capability, offering more raw processing power than the Uno at a similarly low price, though without built-in wireless.

2.2 Basic Sensors

Temperature Sensor

Temperature sensors convert a physical temperature into an electrical signal. Analog sensors such as the LM35 output a voltage proportional to temperature and need one of the microcontroller's analog-to-digital converter (ADC) pins, while digital sensors such as the DHT11/DHT22 or the DS18B20 used in this project perform the analog-to-digital conversion internally and report a ready-to-use digital value over a simple protocol. The DS18B20 used here communicates over a single-wire (1-Wire) digital interface and reports temperature across a range of -55°C to +125°C with ±0.5°C accuracy.

Motion Sensor (PIR)

A Passive Infrared (PIR) sensor detects motion by sensing changes in the infrared radiation given off by warm bodies, such as people or animals, as they move through its field of view. It is widely used for automatic lighting, security and intrusion alarms, and occupancy-based automation.

Light Sensor (LDR)

A Light Dependent Resistor (LDR), or photoresistor, is a sensor whose electrical resistance decreases as the light falling on it increases. It is commonly paired with a simple voltage-divider circuit to trigger automatic street lighting, adjust display brightness, or detect day/night transitions.

3. Mini Project – Smart Temperature Monitor

3.1 Project Overview

The Smart Temperature Monitor is a real-time environmental monitoring system built around the DS18B20 digital temperature sensor and an Arduino Uno. Rather than simply switching a single LED on or off, the system classifies every reading into one of three bands — Safe, Warning, or Danger — and reflects that state instantly through a dedicated LED for each band, escalating to an audible buzzer alarm only once the Danger threshold is crossed. The project was designed and fully tested as a live simulation on Wokwi, an in-browser Arduino/ESP32 simulator, before being documented here.

Key features implemented:

- ✅ Real-time temperature monitoring using the DS18B20 sensor
- ✅ Three-tier status classification — Safe / Warning / Danger
- ✅ Green / Yellow / Red LED indicators for each status
- ✅ Buzzer alarm that activates automatically in the Danger state
- ✅ Live temperature and status readout on the Serial Monitor
- ✅ Maximum temperature recorded since power-on
- ✅ Temperature trend detection (Rising / Falling / Stable)

3.2 Components Required

Component	Qty	Purpose
Arduino Uno	1	Main microcontroller board running the monitoring firmware
DS18B20 Temperature Sensor	1	Measures ambient temperature via a 1-Wire digital interface
Green LED	1	Lights up to indicate SAFE status
Yellow LED	1	Lights up to indicate WARNING status
Red LED	1	Lights up to indicate DANGER status
Active Buzzer	1	Sounds an audible alarm in the DANGER state
220Ω Resistor	3	Current-limiting resistor for each LED
4.7kΩ Resistor	1	Pull-up resistor for the DS18B20 1-Wire data line

3.3 Circuit Diagram & Pin Connections

The DS18B20 uses a single-wire (1-Wire) digital interface, so only one data pin is needed to communicate with the Arduino. Because this bus is open-drain, a 4.7kΩ pull-up resistor is required between the data line and 5V so the line reads HIGH when idle. Each status LED is wired through its own 220Ω current-limiting resistor. The full pin mapping used in this project is listed below, followed by the complete circuit as simulated in Wokwi.

Component	Arduino Pin
DS18B20 – DQ (Data)	D2
DS18B20 – VCC	5V
DS18B20 – GND	GND
4.7kΩ Pull-up Resistor	Between 5V and DQ
Green LED (SAFE) + 220Ω	D5
Yellow LED (WARNING) + 220Ω	D6
Red LED (DANGER) + 220Ω	D7
Active Buzzer	D8

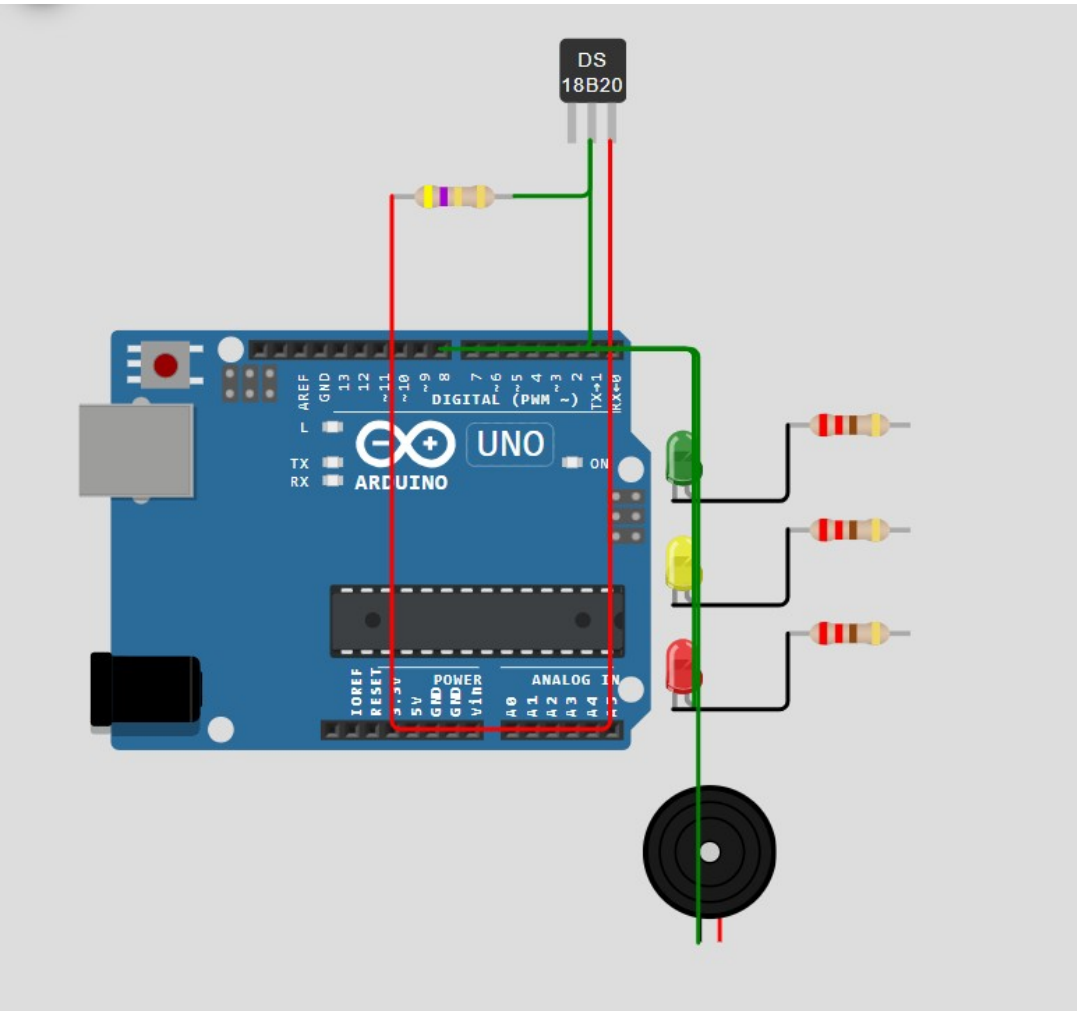

Figure 3.1: Wokwi circuit — Arduino Uno, DS18B20, status LEDs, and buzzer

3.4 Working Principle

The DS18B20 communicates with the Arduino over a single-wire (1-Wire) digital interface, using only one data line (DQ) for both commands and data. Because the 1-Wire bus is open-drain, the line must be pulled to logic HIGH externally; the 4.7kΩ resistor between DQ and the 5V rail performs this pull-up

so the bus rests HIGH when idle, and only the sensor or the microcontroller actively pulls it LOW to communicate.

On every loop, the Arduino issues a “convert temperature” command to the sensor over this bus. The DS18B20 performs the conversion internally and returns a 12-bit digital value, which the DallasTemperature library converts directly into degrees Celsius — removing the need for any analog-to-digital conversion or manual calibration on the Arduino side.

The firmware then compares the reading against two fixed thresholds to classify the environment into one of three bands, each mapped to a dedicated LED so the system's state is visible at a glance even without the Serial Monitor open: SAFE (below 25°C, green LED), WARNING (25°C up to just under 35°C, yellow LED), and DANGER (35°C and above, red LED plus an audible 1kHz buzzer tone). Alongside the live reading, the sketch maintains the highest temperature observed since power-on and reports whether the temperature is rising, falling, or holding steady compared with the previous second's reading — giving a snapshot and a short-term trend without any external logging hardware.

3.5 Source Code (Arduino / C++)

The complete, commented Arduino sketch used in the Wokwi simulation is given below. It uses the OneWire and DallasTemperature libraries to communicate with the DS18B20 over the single-wire bus, and drives the three status LEDs and buzzer directly from digital output pins.

```
/*
  ============================================================
   SMART TEMPERATURE MONITOR PRO
   Embedded Systems Task-1  |  Maincrafts Technology
   Board   : Arduino Uno
   Sensor  : DS18B20 (1-Wire Digital Temperature Sensor)
  ============================================================
*/

#include <OneWire.h>            // 1-Wire communication protocol
#include <DallasTemperature.h>  // Driver for the DS18B20 sensor

#define ONE_WIRE_BUS 2   // DS18B20 data (DQ) pin -> Arduino D2

#define GREEN_LED  5     // SAFE status indicator
#define YELLOW_LED 6     // WARNING status indicator
#define RED_LED    7     // DANGER status indicator
#define BUZZER     8     // Active buzzer for the DANGER alarm

OneWire oneWire(ONE_WIRE_BUS);         // 1-Wire bus on pin D2
DallasTemperature sensors(&oneWire);   // DS18B20 sensor on that bus

float currentTemp;            // Latest temperature reading (deg C)
float previousTemp = -100;    // Previous reading, used for trend; -100 = "no
reading yet"
float maxTemp = -100;         // Highest temperature recorded since power-on

void setup() {
  Serial.begin(9600);      // Start Serial Monitor communication
  sensors.begin();         // Initialize the DS18B20 sensor

  pinMode(GREEN_LED, OUTPUT);
  pinMode(YELLOW_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);
```

```
  Serial.println("================================");
  Serial.println(" SMART TEMPERATURE MONITOR PRO");
  Serial.println("================================");
}

void loop() {
  sensors.requestTemperatures();             // Ask the sensor for a fresh reading
  currentTemp = sensors.getTempCByIndex(0);  // Read temperature in Celsius

  // Handle a disconnected or faulty sensor gracefully
  if (currentTemp == DEVICE_DISCONNECTED_C) {
    Serial.println("Sensor Error!");
    delay(1000);
    return;   // Skip this cycle and try again
  }

  // Track the highest temperature seen so far
  if (currentTemp > maxTemp)
    maxTemp = currentTemp;

  Serial.print("Current Temperature : ");
  Serial.print(currentTemp);
  Serial.println(" C");

  Serial.print("Maximum Temperature : ");
  Serial.print(maxTemp);
  Serial.println(" C");

  // Determine trend by comparing with the previous reading
  // (skipped on the very first loop, since there is no previous reading yet)
  if (previousTemp != -100) {
    if (currentTemp > previousTemp)
      Serial.println("Trend : Rising");
    else if (currentTemp < previousTemp)
      Serial.println("Trend : Falling");
    else
      Serial.println("Trend : Stable");
  }

  // Three-tier status logic: SAFE -> WARNING -> DANGER
  if (currentTemp < 25) {
    Serial.println("Status : SAFE");
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(RED_LED, LOW);
    noTone(BUZZER);
  }
  else if (currentTemp < 35) {
    Serial.println("Status : WARNING");
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(YELLOW_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    noTone(BUZZER);
  }
  else {
    Serial.println("Status : DANGER");
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(RED_LED, HIGH);
```

```
    tone(BUZZER, 1000);   // Sound the buzzer at 1000 Hz
  }

  previousTemp = currentTemp;   // Save this reading for next loop's trend check
  Serial.println("----------------------------");
  delay(1000);   // Wait 1 second before the next reading
}
```

3.6 Code Explanation

The sketch begins by including the OneWire and DallasTemperature libraries and defining the pins used: the DS18B20 data line on D2, the Green / Yellow / Red status LEDs on D5, D6, and D7, and the buzzer on D8.

setup() opens the Serial Monitor at 9600 baud, initializes the DS18B20 sensor, and configures the LED and buzzer pins as OUTPUT before printing a banner to the Serial Monitor.

Inside loop(), the sketch requests a fresh temperature conversion from the sensor every second. It checks for DEVICE_DISCONNECTED_C so a wiring fault is reported as “Sensor Error!” instead of a nonsense temperature value.

The maximum-temperature tracker keeps the highest reading seen since power-on, while the trend comparison (previousTemp vs currentTemp) reports whether the temperature is Rising, Falling, or Stable — skipping the very first loop, since there is no prior reading to compare against yet.

Finally, a three-way if-else block decides the status band and drives the corresponding LED, only sounding the buzzer once the DANGER band (≥35°C) is reached, so the alarm is reserved for genuinely high temperatures.

3.7 Simulation & Testing Results

The finished circuit was tested by adjusting the DS18B20's temperature slider in the Wokwi simulator across all three status bands. The Serial Monitor output and the corresponding LED state were captured for each condition, shown below.

Safe Condition (Temperature < 25°C)

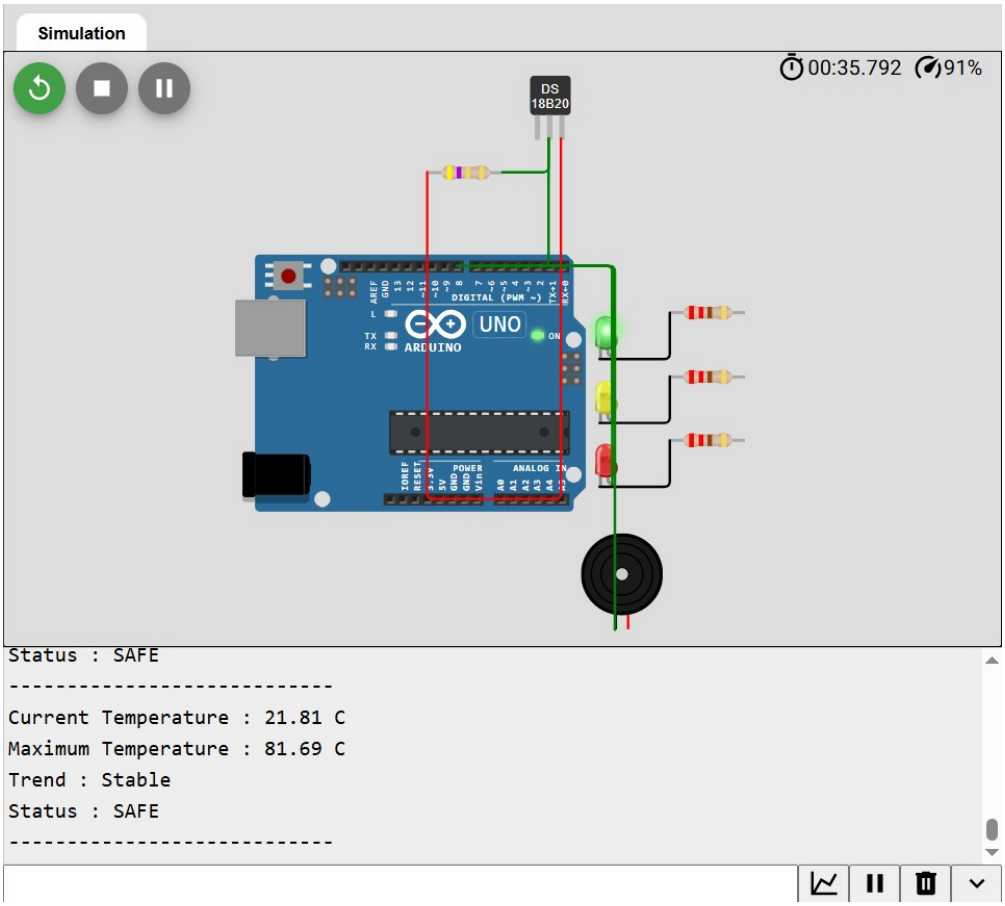

Figure 3.2: SAFE status — Green LED ON, buzzer silent

Current Temperature: 21.81°C | **Maximum Temperature:** 81.69°C | **Trend:** Stable | **Status:** **SAFE**

Warning Condition (25℃ ≤ Temperature < 35℃)

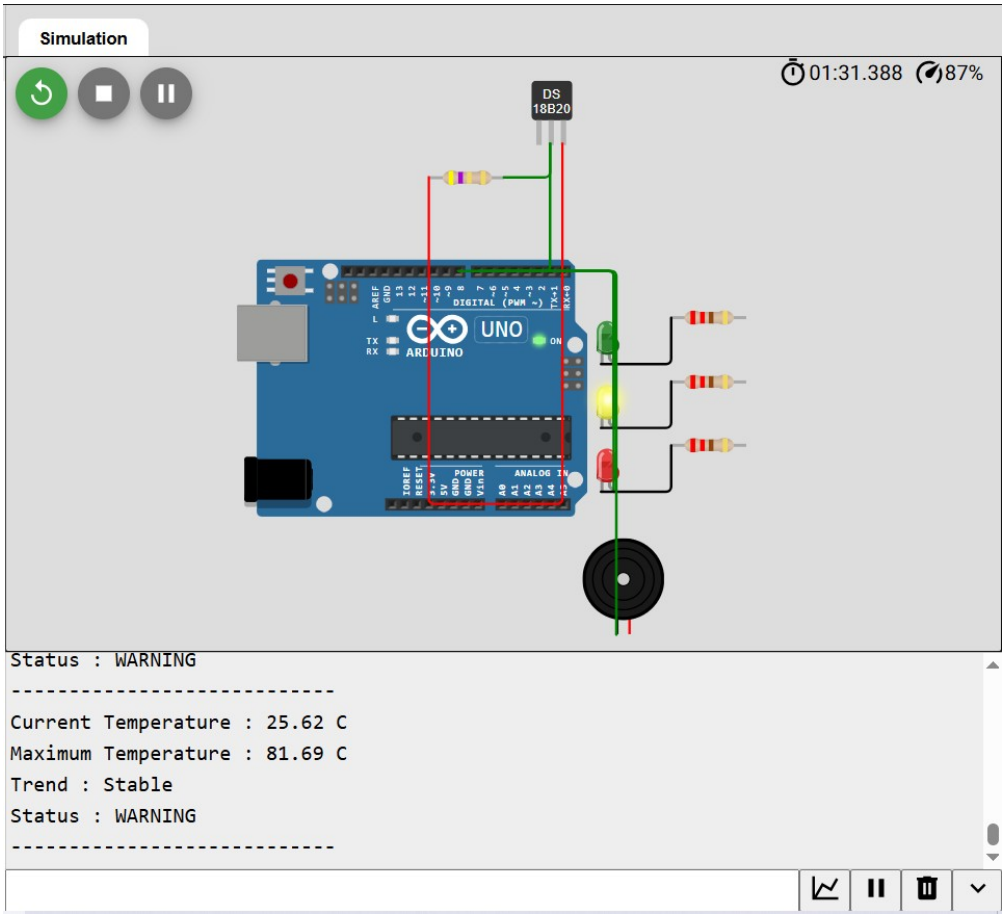

Figure 3.3: WARNING status — Yellow LED ON, buzzer silent

Current Temperature: 25.62℃ | **Maximum Temperature:** 81.69℃ | **Trend:** Stable | **Status:** **WARNING**

Danger Condition (Temperature ≥ 35°C)

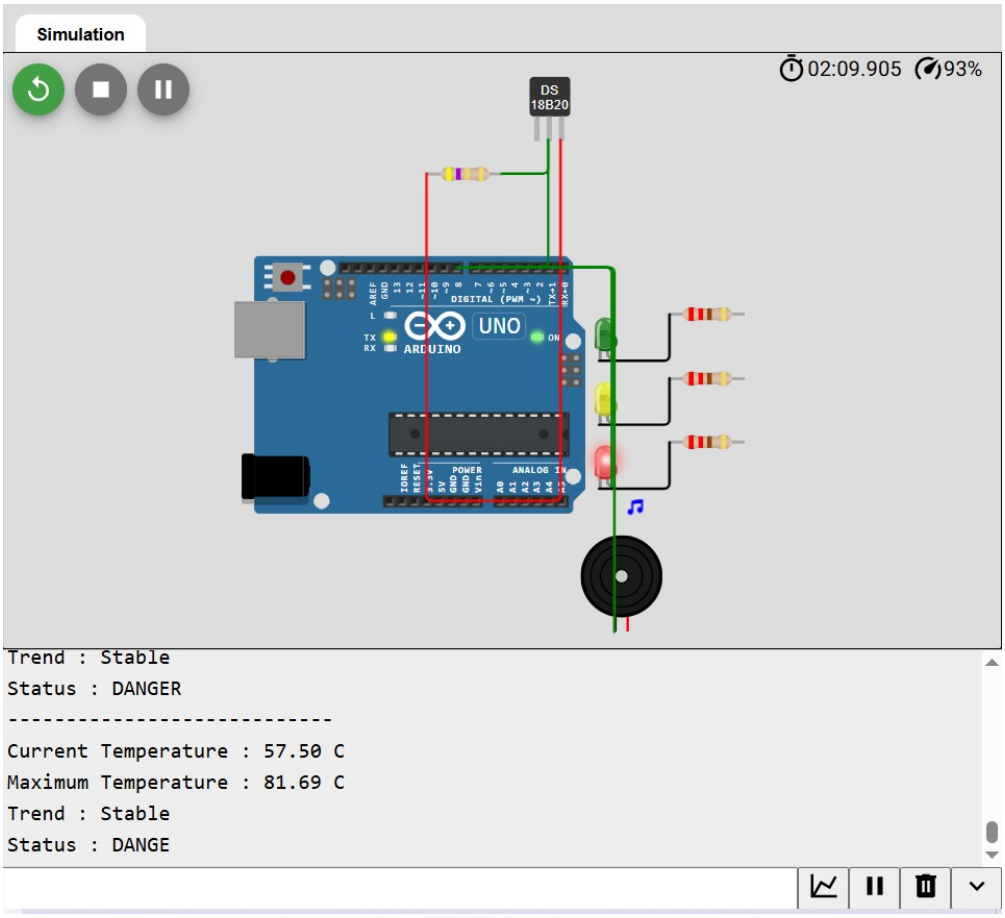

Figure 3.4: DANGER status — Red LED ON, buzzer sounding at 1kHz

Current Temperature: 57.50°C | **Maximum Temperature:** 81.69°C | **Trend:** Stable | **Status:** DANGER

3.8 Live Simulation & Video Demo

The complete project — circuit, firmware, and a working real-time simulation — is hosted publicly on Wokwi and can be opened, run, and edited directly in the browser at the link below. The DS18B20's temperature can be adjusted live from the sensor's pop-up slider to trigger each of the three status bands and watch the LEDs, buzzer, and Serial Monitor respond in real time. A short video walkthrough of the working simulation is also linked below.

- Wokwi Project – Smart Temperature Monitor Pro — https://wokwi.com/projects/471001611597562881
- Video Demo – Smart Temperature Monitor in Action — https://drive.google.com/file/d/1UswRnNJR1L7OnP_HNdfTFN6WjZ7n3ZKz/view?usp=sharing

3.9 Real-World Applications of This Project

- **Server Room / Data Centre Monitoring:** early warning before equipment overheats and hardware is damaged.
- **Cold-Chain & Food Storage:** keeping refrigerated transport and warehouses within a safe temperature band.

- **Industrial Machinery Monitoring:** catching motor or bearing overheating before failure.
- **Greenhouse & Agriculture:** alerting farmers when polytunnel or storage temperatures drift out of range.
- **Laboratory & Pharmaceutical Storage:** protecting temperature-sensitive samples, reagents, and vaccines.
- **Electrical Panel Monitoring:** flagging overheating conductors or connections as an early fire-safety measure.

4. Conclusion

This task provided hands-on exposure to the fundamentals of embedded systems, from the theoretical distinction between microcontrollers and microprocessors to the practical role each major development board plays in real IoT designs. Building the Smart Temperature Monitor consolidated this understanding by requiring the same skills a Junior Embedded Engineer uses on the job: selecting the right sensor, wiring it correctly with the necessary pull-up resistor, writing firmware that reads and interprets sensor data in real time, and communicating the system's state clearly through LEDs, an audible alarm, and a Serial Monitor log. The finished prototype — simulated end-to-end on Wokwi — reads live temperature data, classifies it into Safe, Warning, and Danger bands, tracks the maximum temperature recorded, and reports whether conditions are rising, falling, or stable, giving a small but complete example of the kind of monitoring system embedded engineers build for server rooms, cold storage, and industrial equipment in the real world.

5. References & Resources

- NPTEL – Embedded Systems (IIT Delhi), Prof. Santanu Chaudhury — https://nptel.ac.in/courses/108102045
- Arduino – Official Getting Started Guide — https://docs.arduino.cc/learn/starting-guide/getting-started-arduino/
- Wokwi – Free Online Arduino / ESP32 Simulator — https://wokwi.com/
- Autodesk Tinkercad Circuits – Beginner-friendly browser-based circuit builder — https://www.tinkercad.com/circuits
- This Project – Live Wokwi Simulation (Smart Temperature Monitor Pro) — https://wokwi.com/projects/471001611597562881
- This Project – Video Demo — https://drive.google.com/file/d/1UswRnNJR1L7OnP_HNdfTFN6WjZ7n3ZKz/view?usp=sharing